

Making the Learning Engine Dynamic

A deep-dive technical document explaining how three core components — the SUL Functions, the Observation Table, and the Teacher — were transformed from hard-coded, case-study-specific scripts into a single fully-generic engine driven entirely by user input from the Django web UI.

Component	Old File	New File
SUL Functions	case_studies/<cs>/sul_functions.py	core_algorithm/dynamic_sul.py
Observation Table	Isha/.../obstable.py	core_algorithm/dynamic_obstable.py
Teacher	Isha/.../teacher.py	core_algorithm/teacher.py (CustomTeacher)

PART 1 — Dynamic SUL Functions

The **System Under Learning (SUL)** is the software layer that sits between the raw trace data and the LSHA learning algorithm. It answers three questions: *Where are the change points in a trace? What event label applies at each point? What physics parameter characterises each segment?*

In the original project these three questions were answered by a separate **sul_functions.py** file inside each case-study folder (thermostat, HRI, energy_sim, gr3n). Each file contained hard-coded variable names, threshold constants, and physics formulas specific to that experiment. Adding a new case study required writing a brand-new file from scratch.

1.1 What the old sul_functions.py files looked like

Here is a concrete example. The **energy_sim** case study hard-codes three constants from config.ini and uses them directly in every function:

```
# energy_sim/sul_functions.py (OLD) SPEED_RANGE = int(config['ENERGY CS']['SPEED_RANGE'])
# = 200 MIN_SPEED = int(config['ENERGY CS']['MIN_SPEED']) # = 100 MAX_SPEED =
int(config['ENERGY CS']['MAX_SPEED']) # = 10000 def is_chg_pt(curr, prev): # 'curr' is a
2-tuple: (speed_value, pressure_value) return abs(curr[0] - prev[0]) > SPEED_RANGE or
curr[1] != prev[1] def label_event(events, signals, t): speed_sig = signals[1] #
hard-coded index! pressure_sig = signals[2] # hard-coded index! # ... builds intervals
from MIN_SPEED / SPEED_RANGE / MAX_SPEED ... for i, interval in
enumerate(SPEED_INTERVALS): if interval[0] <= curr_speed < interval[1]: identified_event
= events[i] # picks from hard-coded list def parse_data(path): # reads CSV columns by
FIXED position: row[2]=time, row[3]=speed, row[4]=power ts = parse_ts(row[2]) speed_v =
round(float(row[3]) / 100) * 100 power_v = float(row[4])
```

The **thermostat** sul_functions.py is equally rigid — it hard-codes signal indices (signals[0]=heater, signals[1]=temperature, signals[2]=window), reads ON_R=100.0 as a literal constant, and branches on CS_VERSION 1-10 with explicit if/else chains.

If a user wanted to try a machine with different speed thresholds, or rename a column, or add a third signal, they had to edit Python source files. That is exactly what the dynamic approach eliminates.

1.2 What the user provides instead

Instead of editing code, the user fills in a web form. The key fields are:

Django field	Example value	What it controls
resample_strategy	"UPPAAL" or "SIM"	Which data source to use
main_variable	"T_r"	Column/signal that is the main observable
driver_signal	"t.ON"	Column that causes discrete state changes
context_variables	["r.open"]	Extra columns available to guards
user_json	{models, events, trace_generation, ...}	Full physics + event definition

The **user_json** field is a JSON text area. For the thermostat V1 it looks like:

```
{ "models": [ { "id": 0, "name": "Cooling", "type": "EXP_DECAY" }, { "id": 1, "name":
"Heating", "type": "EXP_GROWTH" } ], "events": [ { "symbol": "h_0", "guard": "t.ON ==
1.0", "model_id": 1, "trigger_value": 0 }, { "symbol": "c_0", "guard": "t.ON == 0.0",
"model_id": 0, "trigger_value": 0 } ], "trace_generation": { "xml_force_variable":
"force_open", "xml_action_variable": "force_exe", "max_length": 15 } }
```

1.3 How `dynamic_sul.py` is structured

The new file has one public entry point for each of the three SUL questions. Everything else is a private helper:

Function	Question it answers
<code>parse_data_dynamic(file_paths, args)</code>	Load trace file(s) → signals dictionary
<code>is_chg_pt_dynamic(signals, index, args)</code>	Is index a change point?
<code>label_event_dynamic(signals, index, args)</code>	Which event symbol is active at index?
<code>get_physics_param_dynamic(signals, s, e, args)</code>	What rate/mean characterises segment [s..e]?

The **args** dictionary passed to every function is assembled in `tasks.py` from the Django model:

```
sul_args = { 'resample_strategy': cs_instance.resample_strategy, 'main_var':
cs_instance.main_variable, 'driver': cs_instance.driver_signal, 'context_variables':
cs_instance.context_variables, 'events': data_dict['events'], # from user_json 'models':
data_dict['models'], # from user_json 'trace_generation': data_dict['trace_generation'],
}
```

1.4 Change 1 — `parse_data_dynamic`: generic trace parser

What it replaced

Every old `sul_functions.py` had its own **`parse_data(path)`** that hard-coded the file format. The energy version read CSV columns by numeric index. The thermostat version parsed a UPPAAL text format by searching for known variable name headers. You could not reuse either for any other case study.

How the new version works — routing by strategy

`parse_data_dynamic` checks **`args['resample_strategy']`** (or `trace_generation.strategy`) and routes to one of two private parsers:

```
strategy = trace_config.get('strategy') or args.get('resample_strategy', 'UPPAAL') if
strategy == 'SIM': return _parse_csv(file_paths, target_vars, trace_config) else: return
_parse_uppaal(file_paths[0], target_vars)
```

`_build_target_vars` — the column-to-key map

Both parsers need to know which columns to extract and how to name them internally. This map is built once from the user's fields:

```
# User set: main_variable='T_r', driver_signal='t.ON', context=['r.open'] target_vars = {
'T_r' : 'main', # main observable always mapped to 'main' 't.ON' : 't.ON', # driver keeps
its original name 'r.open' : 'r.open', # context variable keeps its name }
```

The output signals dictionary always has this shape, no matter which case study or how many variables there are:

```
signals = { 'time' : np.array([0.0, 1.0, 2.0, ...]), # seconds from start 'main' :
np.array([15.2, 15.04, 14.89, ...]), # T_r values 't.ON' : np.array([0.0, 0.0, 0.0,
...]), # heater state 'r.open' : np.array([0.0, 0.0, 0.0, ...]), # window state }
```

✓ The key insight: downstream code (`is_chg_pt`, `label_event`, `get_physics_param`) never needs to know column positions. It only uses dictionary keys — keys that the user defined.

`_parse_csv` — reading the energy / SIM case studies

The CSV parser applies a fixed six-step preprocessing pipeline driven entirely by the `trace_generation` sub-object in `user_json`:

Step	What happens	Config key used
1. Load	Read all uploaded CSV files and concatenate	—
2. Pivot	If vertical format, pivot rows to columns	<code>csv_format</code> , <code>key_column</code> , <code>value_column</code>
3. Replace	Convert string tokens to numbers	<code>replace_values</code> : {"OFF": 0, "ON": 1}
4. Time	Parse timestamp → elapsed seconds	<code>time_column</code> , <code>time_format</code>
5. Interpolate	Fill NaN gaps (sensors report at different rates)	<code>interpolate_method</code>
6. Round	Discretise continuous signals to buckets	<code>round_columns</code> : {"RPM": 200}

Example for the energy CSV files (W7/W9 data): The column `HEADSTOCK__SPINDLE_MOTOR__1__RPM` contains continuous values like 2285.4, 2285.8, 2286.1. After step 6 with bucket size 200, all three become 2200. Without rounding, every tiny floating-point wobble would look like a new event. With rounding, the signal is cleanly discrete.

[_parse_uppaal — reading the thermostat / HRI trace files](#)

UPPAAL verifyta produces a plain text file where each variable has a header line followed by space-separated (time, value) pairs. The parser:

- Reads variable name headers (lines ending with ':')
- Normalises whitespace in names so `'amy.busy || amy.p_2'` matches `'amy.busy||amy.p_2'`
- Ignores `[1]:`, `[2]:` lines (duplicate simulation runs caused by UPPAAL deadlocks)
- Forward-fills values onto a unified time axis (UPPAAL hold-last-value semantics)

1.5 Change 2 — `is_chg_pt_dynamic`: generic change-point detection

What the old version did

Each case study had its own `is_chg_pt` with hardcoded logic. Energy:

```
def is_chg_pt(curr, prev): return abs(curr[0] - prev[0]) > SPEED_RANGE or curr[1] != prev[1] # curr[0] is always speed, curr[1] is always pressure – fixed positions!
```

Thermostat:

```
def is_chg_pt(curr, prev): return curr != prev # checks the entire 3-tuple for any change
```

What the new version does — two independent checks

The new function applies two checks in order. Either one returning True declares a change point:

```
# CHECK 1 – Did the event label change? curr_label = label_event_dynamic(signals, index, args) prev_label = label_event_dynamic(signals, index - 1, args) if curr_label != prev_label: return True # e.g. heater just turned on: c_0 -> h_0 # CHECK 2 – Did any driver signal step beyond its tolerance? for driver_name in drivers: # e.g. ['t.ON'] or ['RPM'] curr_val = signals[driver_name][index] prev_val = signals[driver_name][index - 1] tol = tolerances.get(driver_name, 0) # from physics_constants if abs(curr_val - prev_val) > tol: return True
```

Where the tolerance comes from: the user puts it inside `trace_generation.physics_constants.tolerances` in `user_json`:

```
"physics_constants": { "MIN_SPEED": 100, "tolerances": {
"HEADSTOCK__SPINDLE_MOTOR__1__RPM": 200 } }
```

For thermostat, t.ON only takes values 0 and 1, so any change is significant — tolerance is 0 (the default). For energy RPM, a tolerance of 200 means small floating-point wobble around 2285 RPM does not trigger a change point; only a jump bigger than 200 does.

1.6 Change 3 — label_event_dynamic: guard-based event identification

What the old version did

The old label_event functions contained chains of if/elif statements that checked hard-coded signal indices and constants. HRI had nested conditions across CS_VERSION and SAMPLE_STRATEGY. Thermostat branched on CS_VERSION 1-10. None of this logic was reusable.

What the new version does — evaluate guard strings

The user defines guards in the events array of user_json. The function evaluates them in order and returns the symbol of the first one that matches:

```
# Build the evaluation context context = {} for key, arr in signals.items(): context[key]
= arr[index] # current value context[f'prev_{key}'] = arr[index - 1] # previous value
context.update(physics_constants) # e.g. MIN_SPEED=100 # Evaluate each event guard in
definition order for event_def in args['events']: if safe_eval(event_def['guard'],
context): return event_def['symbol']
```

safe_eval — how guards are evaluated securely

Guards are Python-style boolean expressions supplied by the user as strings. Calling eval() on user input is dangerous, so safe_eval sandboxes it:

- No Python built-ins are available (no __import__, no open, no exec)
- Only the math module and the explicitly provided context variables are accessible
- Variable names are replaced by positional placeholders (__VAR_0__, __VAR_1__, ...) before eval() is called, sorted by descending length to prevent shorter names from accidentally matching inside longer ones

Example — thermostat guard 't.ON == 1.0': The context has the key 't.ON' (a string with a period). safe_eval replaces the literal substring 't.ON' with '__VAR_0__' in the expression, giving '__VAR_0__ == 1.0'. Python evaluates this cleanly as a simple comparison. t.ON would have been illegal as a Python identifier, but the replacement happens at the string level before eval ever sees it.

Example — HRI guard 'amy.busy || amy.p_2 == 1.0': The signal key is 'amy.busy || amy.p_2' (the full UPPAAL expression, 20 characters). Because keys are sorted longest-first, the whole substring 'amy.busy || amy.p_2' is replaced with '__VAR_0__' before eval. The result '__VAR_0__ == 1.0' is valid Python. Without the replacement, Python would misparse '||' as a syntax error; with it, the replacement absorbs both the name and the operator.

Example — energy guard with symbolic constants:

```
guard: "MIN_SPEED <= HEADSTOCK_SPINDLE_MOTOR__1__RPM and
HEADSTOCK_SPINDLE_MOTOR__1__RPM < 1000" context['MIN_SPEED'] = 100 # from
physics_constants context['HEADSTOCK_SPINDLE_MOTOR__1__RPM'] = 600 # current RPM #
After safe_eval replacement: # '__VAR_1__ <= __VAR_0__ and __VAR_0__ < 1000' # Evaluates
to: 100 <= 600 and 600 < 1000 -> True -> returns 'e_low'
```

1.7 Change 4 — get_physics_param_dynamic: model-aware parameter extraction

What the old versions did

Each case study had a dedicated parameter function with a completely different formula hard-coded for its specific physics:

- thermostat: `get_thermo_param` — exponential heating/cooling with $R=100$ and K constants
- energy: `get_power_param` — arithmetic mean of power values
- HRI: `get_ftg_param` — log-ratio formula for fatigue growth and recovery

None of these formulas were shared. Adding a new case study meant inventing and coding a new formula.

What the new version does — model-type dispatch

The new function looks up the model type from the user's JSON and dispatches to the appropriate formula. The model type for the active segment is found by:

- Calling `label_event_dynamic` to get the event symbol at `start_i`
- Finding that symbol in `args['events']` to get its `model_id`
- Finding that `model_id` in `args['models']` to get its type string

```
# User JSON says: # events: [{symbol:'c_0', model_id:0}, {symbol:'h_0', model_id:1}] #  
models: [{id:0, type:'EXP_DECAY'}, {id:1, type:'EXP_GROWTH'}] # At a segment starting  
during cooling (c_0 is active): symbol = 'c_0' # from label_event_dynamic model_id = 0 #  
from events lookup type = 'EXP_DECAY' # from models lookup
```

Then `_compute_rate` applies the right formula for that type:

Model type	Formula	Captures	Example
MEAN	arithmetic mean	Average value in segment	Power: 2575 W
EXP_DECAY	$\text{mean}(-\log(v[i]/v[i-1]))$	Decay constant R	Cooling: $R \approx 0.01$
EXP_GROWTH	$\text{mean}(v[i] - v[i-1])$	Growth increment K	Heating: $K \approx 0.55$ °C/step
LINEAR_DECAY	$\text{mean}(v[i-1] - v[i])$	Slope α	Custom: 0.3 per second
LINEAR_GROWTH	$\text{mean}(v[i] - v[i-1])$	Slope α	Custom: 0.3 per second

The function always returns both mean and rate so that the adapter in `tasks.py` can use whichever is appropriate for the configured case study.

PART 2 — Dynamic Observation Table

The **Observation Table (ObsTable)** is the central data structure of the L* learning algorithm. It is a two-part matrix: the upper half (S set) contains the candidate prefixes for automaton locations; the lower half (low_S) contains extensions being tested. Each cell holds the physics state (flow model + probability distribution) observed when the system was run with that word.

The table is also responsible for two structural operations: **to_sha** converts the table into a Stochastic Hybrid Automaton (the learned result), and **get_loc_from_word** maps any word to its automaton location by comparing rows.

2.1 The single hardcoded parameter: EQ_CONDITION

The original `obstable.py` had exactly one parameter that controlled behaviour, and it was baked in at import time:

OLD (hard-coded)	NEW (dynamic)
<pre>import configparser config.read('config.ini') EQ_CONDITION = config['LSHA PARAMETERS'] ['EQ_CONDITION'].lower() # Module-level constant — # fixed for the entire process</pre>	<pre># No configparser, no .ini file # No module-level constant # Read live inside each method: eq_condition = teacher.config .get('eq_condition', 's').lower()</pre>

Removing the `config.ini` dependency from `obstable.py`

EQ_CONDITION controls whether two rows in the table are considered equivalent (same automaton location) using strict or weak equality:

Value	Meaning	Effect on the learned automaton
"s" (strict)	Rows must match exactly in every filled cell	Fewer merged locations; may produce more states but is safer
"w" (weak)	Rows match if no filled cell explicitly disagrees	More aggressive merging; automaton may be non-deterministic but

2.2 Where `eq_condition` now comes from — the data flow

The value travels from the user's browser to every row comparison inside the observation table through the following chain:

```
# Step 1: User selects 's' or 'w' in the React web UI # Django saves it to:
CaseStudy.eq_condition (CharField) # Step 2: tasks.py builds the teacher config dict:
teacher_config = { 'eq_condition': getattr(cs_instance, 'eq_condition', 's'), ... #
noise, p_value, ht_query, n_min, etc. } # Step 3: CustomTeacher stores it: class
CustomTeacher: def __init__(self, sul, trace_generator, config_data): self.config =
config_data # the whole dict self.eq_condition = self.config.get('eq_condition',
's').lower() # Step 4: dynamic_obstable.py reads it at call time: def
get_loc_from_word(self, word, locations, seq_to_loc, teacher): eq_condition =
teacher.config.get('eq_condition', 's').lower() ... if eq_condition == 's':
teacher.eqr_query(curr_row, row, strict=True) else: teacher.eqr_query(curr_row, row,
strict=False)
```

■ In the old version, two users running different case studies with different `eq_condition` settings in the same Python process would both get the same value because `EQ_CONDITION` was a module-level singleton set at import time. Now each call reads from the teacher object, which was created fresh for that specific case study run.

2.3 Where eq_condition is used inside the table — three locations

Location A — get_loc_from_word (edge routing)

When building the SHA edges, every word must be mapped to an automaton location. The function walks the upper observations and asks: 'Is the row for this word equivalent to any row that already has a named location?' The equivalence check uses eq_condition.

```
eq_condition = teacher.config.get('eq_condition', 's').lower() for i, row in
enumerate(self.get_upper_observations()): if eq_condition == 's': match =
teacher.eqr_query(curr_row, row, strict=True) else: match = teacher.eqr_query(curr_row,
row, strict=False)
```

Location B — to_sha (location creation)

When converting the table to an automaton, the method groups rows into unique locations. Two rows that are 'equivalent' become the same location. Again eq_condition drives the grouping:

```
eq_condition = teacher.config.get('eq_condition', 's').lower() for i, row in
enumerate(upp_obs): for seq in unique_sequences: row_2 = upp_obs[self.get_S().index(seq)]
if eq_condition == 's': already_present = teacher.eqr_query(row, row_2, strict=True)
else: already_present = teacher.eqr_query(row, row_2, strict=False)
```

2.4 The hard-coded flow label — the most impactful fix

This is the change that actually broke every non-HRI case study in the old version. The flow label is the text written on each automaton location — it describes the physics that govern the system while in that state (e.g. 'cooling R=0.01').

OLD (hard-coded)	NEW (dynamic)
<pre># in to_sha: new_flow = (row.state[0].vars[0][0].label + ', ' + row.state[0].vars[0][1].label) # Assumes EXACTLY 2 physics # variables: vars[0][0] and # vars[0][1]. # Crashes for Energy (1 var), # Thermostat (1 var), and any # custom case study.</pre>	<pre>try: if hasattr(row.state[0], 'vars') and isinstance(..., (list,tuple)): flat_vars = [] for v in row.state[0].vars: if isinstance(v, (list,tuple)): flat_vars.extend(v) else: flat_vars.append(v) new_flow = ', '.join([getattr(v,'label',str(v)) for v in flat_vars]) except Exception: new_flow = str(row.state[0])</pre>

Flow label construction: hard-coded 2-variable access vs. generic flatten

What vars[0] contains depends entirely on the case study. For HRI it is [(lambda_var, mu_var)] — a list containing a tuple of two variables. For Energy it is [mean_var] — a list with one variable. For a custom 3-variable case study it would be [(v1, v2, v3)].

The new code flattens whatever structure exists and joins all variable labels with commas. Example outputs:

Case study	vars structure	Old result	New result
HRI	[(lambda, mu)]	lambda, mu ✓	lambda, mu ✓
Energy	[mean_power]	IndexError ✗	mean_power ✓
Thermostat	[R]	IndexError ✗	R ✓
Custom 3-var	[(v1, v2, v3)]	IndexError ✗	v1, v2, v3 ✓

2.5 The safe max() fix — preventing crashes on empty tables

At the very start of the L* algorithm, before any observations have been collected, the observation table is empty. If the `__str__` method is called at this point (e.g. for logging), the old code crashed:

OLD (hard-coded)	NEW (dynamic)
<pre>max_tabs = max([len(str(word)) for i, word in enumerate(...) if i in populated_rows]) # If populated_rows is empty, # max([]) raises ValueError: # 'max() arg is an empty sequence'</pre>	<pre>valid_lens = [len(str(word)) for i, word in enumerate(...) if i in populated_rows] max_tabs = max(valid_lens) if valid_lens else 8 # Returns default width 8 # when the table is empty.</pre>

Safe max() with default fallback in `__str__`

PART 3 — Dynamic Teacher (CustomTeacher)

The **Teacher** is the orchestrator of the L* learning loop. It is responsible for four types of queries:

Query	Abbreviation	What it does
Model Identification	mi_query	Fits a physics model (EXP_DECAY etc.) to a segment
Hypothesis Testing	ht_query	Assigns a probability distribution to a flow condition
Row Equality	eqr_query	Checks if two table rows represent the same state
Counterexample	get_counterexample	Finds a word that proves the table is not closed or consistent

In the original project, the Teacher was a single class in `lsha/sha_learning/learning_setup/teacher.py` that read ALL its configuration from `config.ini` at module import time. This made it impossible to run different case studies with different settings without manually editing the `.ini` file.

3.1 How the old Teacher was configured

The old file had a block of module-level constants that were read once and never changed during the lifetime of the Python process:

```
# lsha/sha_learning/learning_setup/teacher.py (OLD)
config = configparser.ConfigParser()
config.read('...sha_learning/resources/config/config.ini')
CS = config['SUL CONFIGURATION']['CASE_STUDY'] # e.g. 'THERMO'
NOISE = float(config['LSHA PARAMETERS']['DELTA']) # e.g. 1.0
P_VALUE = 0.0 # hard-coded!
MI_QUERY = config['LSHA PARAMETERS']['MI_QUERY'] == 'True'
PLOT_DDTW = config['LSHA PARAMETERS']['PLOT_DDTW'] == 'True'
HT_QUERY = config['LSHA PARAMETERS']['HT_QUERY'] == 'True'
HT_QUERY_TYPE = config['LSHA PARAMETERS']['HT_QUERY_TYPE'] # 'D' or 'S'
EQ_CONDITION = config['LSHA PARAMETERS']['EQ_CONDITION'].lower()

class Teacher:
    def __init__(self, sul, pov=None, start_dt=None, ...):
        self.sul = sul
        self.TG = TraceGenerator(pov=pov, ...)
        self.hist = {}
```

Every method then used these module-level variables directly. For example `mi_query` hard-coded a special case for the thermostat:

```
def mi_query(self, word):
    if not MI_QUERY or word == '':
        return
    self.flows[0][self.sul.default_m] ...
    if CS == 'THERMO' and word[-1].symbol == 'h_0':
        return self.flows[0][2] # hard-coded index for thermostat model!
```

■ *P_VALUE was even hard-coded as 0.0 in the module, not read from config. This meant the statistical test threshold could never be changed without editing the source code.*

3.2 How the new CustomTeacher is configured

The new `CustomTeacher` receives a **config_data dictionary** at construction time. This dictionary is built in `tasks.py` directly from the Django database model, so every field reflects exactly what the user set in the web UI:

```
# tasks.py
teacher_config = {
    'noise': getattr(cs_instance, 'noise', 0.0),
    'p_value': getattr(cs_instance, 'p_value', 0.05),
    'mi_query': getattr(cs_instance, 'mi_query', False),
    'plot_ddtw': getattr(cs_instance, 'plot_ddtw', False),
    'ht_query': getattr(cs_instance, 'ht_query', False),
    'ht_query_type': getattr(cs_instance, 'ht_query_type', 'D'),
    'eq_condition': getattr(cs_instance, 'eq_condition', 's'),
    'n_min': getattr(cs_instance, 'n_min', 10),
    'is_aggregation': getattr(cs_instance, 'is_aggregation', False),
}
teacher = CustomTeacher(sul=sul, config_data=teacher_config, trace_generator=custom_tg)
```

Inside CustomTeacher.__init__, each value is stored as an instance attribute and logged so the user can verify their settings took effect:

```
class CustomTeacher:
    def __init__(self, sul, trace_generator=None, config_data=None):
        self.config = config_data if config_data else {}
        self.noise = float(self.config.get('noise', 0.0))
        self.p_value = float(self.config.get('p_value', 0.05))
        self.mi_query_flag = self.config.get('mi_query', False)
        self.plot_ddtw = self.config.get('plot_ddtw', False)
        self.ht_query_flag = self.config.get('ht_query', False)
        self.ht_query_type = self.config.get('ht_query_type', 'D')
        self.eq_condition = str(self.config.get('eq_condition', 's')).lower()
        self.n_min = int(self.config.get('n_min', 10))
        self.is_aggregation = self.config.get('is_aggregation', False)
        self.TG = trace_generator # not built internally!
        self.hist = {}
```

3.3 Change-by-change breakdown

Change A — All module-level constants removed

Every constant from the old teacher.py top block (CS, NOISE, P_VALUE, MI_QUERY, PLOT_DDTW, HT_QUERY, HT_QUERY_TYPE, EQ_CONDITION) is now an instance attribute set from config_data. The configparser import is also gone.

Old constant	Old source	New attribute	Django field
CS	config.ini	— (removed)	Case study name is just a label now
NOISE	DELTA in .ini	self.noise	CaseStudy.noise (FloatField)
P_VALUE	hard-coded 0.0	self.p_value	CaseStudy.p_value (FloatField)
MI_QUERY	config.ini	self.mi_query_flag	CaseStudy.mi_query (BooleanField)
PLOT_DDTW	config.ini	self.plot_ddtw	CaseStudy.plot_ddtw (BooleanField)
HT_QUERY	config.ini	self.ht_query_flag	CaseStudy.ht_query (BooleanField)
HT_QUERY_TYPE	config.ini	self.ht_query_type	CaseStudy.ht_query_type (CharField)
EQ_CONDITION	config.ini	self.eq_condition	CaseStudy.eq_condition (CharField)

Change B — mi_query: removed the CS='THERMO' special case

OLD (hard-coded)	NEW (dynamic)
<pre>def mi_query(self, word): if not MI_QUERY or word == '': return self.flows[0][self.sul.default_m] ... # HARD-CODED THERMOSTAT BRANCH: if CS == 'THERMO' and \ word[-1].symbol == 'h_0': return self.flows[0][2] ...</pre>	<pre>def mi_query(self, word): if not self.mi_query_flag \ or word == '': return self.flows[0][self.sul.default_m] ... # No CS-specific branch. # The model at index 2 comes # from the user's models list, # not a hard-coded number.</pre>

mi_query: removing the CS='THERMO' hard-coded model index

The old code forced the thermostat to always use model index 2 for event h_0. This was correct for the specific thermostat version used in the research paper but would silently produce wrong results for any other case study or any thermostat variant where h_0 mapped to a different model. The new code simply trusts the model index declared in the user's JSON.

Change C — ht_query: routing by self.ht_query_type instead of HT_QUERY_TYPE

OLD (hard-coded)	NEW (dynamic)
<pre>def ht_query(self, word, flow, save=True): if not HT_QUERY or word == '': return self.distributions [self.sul.default_d] if HT_QUERY_TYPE == 'D': return self.ht_d_query(...) else: return self.ht_s_query(...)</pre>	<pre>def ht_query(self, word, flow, save=True): if not self.ht_query_flag \ or word == '': return self.distributions [self.sul.default_d] if self.ht_query_type == 'D': return self.ht_d_query(...) else: return self.ht_s_query(...)</pre>

ht_query: switching from module constants to instance attributes

What 'D' vs 'S' means: HT_QUERY_TYPE='D' (Deterministic) uses exact parameter matching — two segments are considered the same flow condition only if their physics parameters are numerically identical. HT_QUERY_TYPE='S' (Stochastic) uses the Kolmogorov-Smirnov test — two segments are the same if their parameter distributions cannot be statistically distinguished. The user chooses which is appropriate for their system via the web UI's 'HT Query Type' dropdown.

Change D — ht_s_query: NOISE and P_VALUE replaced by instance attributes

The stochastic hypothesis test contains two hardcoded decisions from the old code: the noise level added to the comparison samples, and the statistical significance threshold. Also, the thermostat case study was handled specially (no noise for THERMO).

OLD (hard-coded)	NEW (dynamic)
<pre># OLD noise handling: if CS == 'THERMO': v1 = metrics noise1 = [0] * len(v1) else: v1 = [avg] * 50 noise1 = np.random.normal(0.0, NOISE, size=len(v1)) # OLD significance threshold: if pvalue >= P_VALUE: # was 0.0 best_fit = fits[0]</pre>	<pre># NEW noise handling (is_aggregation # field replaces CS=='THERMO'): if not self.is_aggregation: v1 = metrics noise1 = [0] * len(v1) else: v1 = [avg] * 50 noise1 = np.random.normal(0.0, self.noise, size=...) # NEW significance threshold: if pvalue >= self.p_value: best_fit = fits[0]</pre>

ht_s_query: replacing CS-specific branch and hard-coded P_VALUE=0.0 with UI values

What is aggregation means: When is_aggregation=False (the default), each observed metric is used directly as a data point in the KS test — no noise, no expansion. This is appropriate when the data is deterministic or the user wants exact matching. When is_aggregation=True, metrics are expanded to 50 samples with Gaussian noise of standard deviation self.noise. This simulates stochastic variation and is suitable for noisy physical systems like HRI fatigue measurement. Previously this choice was implicit in whether CS was 'THERMO'.

What p_value controls: The KS test compares two distributions and returns a p-value. If the p-value is above self.p_value, the two distributions are considered indistinguishable and the same probability distribution is reused. A higher p_value threshold is more permissive (merges more distributions). The old code hard-coded this as 0.0, meaning distributions were NEVER merged — every segment got a new distribution. The new default of 0.05 is a statistically conventional choice.

Change E — ref_query: n_min from self.n_min; CSV vs UPPAAL path split

OLD (hard-coded)	NEW (dynamic)
------------------	---------------

<pre>def ref_query(self, table): n_resample = int(config['LSHA PARAMETERS'] ['N_min']) ... for word in uq: self.TG.set_word(word + e) path = self.TG.get_traces(n_resample) if path is not None: for sim in path: self.sul .process_data(sim)</pre>	<pre>def ref_query(self, table): n_resample = int(self.n_min) ... for word in uq: self.TG.set_word(word + e) path = self.TG.get_traces(n_resample) if path and len(path) > 0: if TG.resample_strategy=='CSV': self.sul.process_data(path) else: for sim in path: self.sul.process_data(sim) # empty list -> do nothing</pre>
---	---

ref_query: n_min from instance + CSV vs UPPAAL path handling

What n_min controls: When the teacher refines its knowledge, it resamples ambiguous words this many times. A higher n_min gives more statistically reliable estimates but takes longer. The user sets this in the 'N Min' field of the web UI.

Why the CSV path is different: For UPPAAL, get_traces() returns a list of file paths — one per simulation run. Each file must be processed individually because each is an independent trace. For CSV, get_traces() returns the list of ALL uploaded CSV files at once. These must be passed together so the parser can concatenate them and produce one unified time series. Passing them one at a time would give partial data with gaps at every file boundary.

Why an empty list is silently ignored: The CustomTraceGenerator uses a csv_yielded flag. After yielding the CSV files once, every subsequent call to get_traces() returns an empty list. This prevents the ref_query loop from running forever. The check 'if path and len(path) > 0' cleanly handles this without any error or log spam.

Change F — not_closed and not_consistent: EQ_CONDITION removed

OLD (hard-coded)	NEW (dynamic)
<pre>def not_closed(self, table, new_row): if EQ_CONDITION == 's': eq_rows = [r for r in ... self.eqr_query(..., strict=True)] else: eq_rows = [r for r in ... self.eqr_query(..., strict=False)] # Also in not_consistent: if EQ_CONDITION == 's' and ... elif EQ_CONDITION == 'w' and ...</pre>	<pre>def not_closed(self, table, new_row): is_strict = (self.eq_condition == 's') eq_rows = [r for r in ... self.eqr_query(new_row, r, strict=is_strict)] # Also in not_consistent: is_strict = (self.eq_condition == 's') equal = self.eqr_query(..., strict=is_strict) # Single branch – no if/elif</pre>

not_closed / not_consistent: collapsing the double if/elif into a boolean flag

Beyond removing the global constant, the new code also collapses the if EQ_CONDITION=='s' / elif EQ_CONDITION=='w' double branches into a single boolean is_strict = (self.eq_condition == 's'). This reduces code duplication and makes it immediately obvious what the flag controls.

Change G — get_counterexample: removed the CS-specific ENERGY branch

OLD (hard-coded)	NEW (dynamic)
<pre># End of get_counterexample: if CS in ['ENERGY', 'AUTO_TWIN'] and len(not_counter) > 0: new_events = set([e.symbol for x in not_counter for e in x.events]) - set([e.symbol for t in S for e in t.events]) if len(new_events) > 0: return not_counter[-1] else: return None</pre>	<pre># get_counterexample remains # structurally the same but # the CS=='ENERGY' special case # is gone. # The standard L* algorithm # handles all case studies. # No hard-coded exceptions # for specific system names. return None</pre>

get_counterexample: removing the ENERGY/AUTO_TWIN special-case branch

The old code had a branch that returned a counterexample only for the ENERGY and AUTO_TWING case studies when new event symbols were found in traces that were not yet in the observation table. This was a workaround for a specific behaviour of those case studies, not a general algorithm property. Removing it makes the teacher apply the same L* logic to every case study.

SUMMARY — Complete picture of what changed and why

The table below maps every removed constant and hard-coded value to the user field that replaced it, and shows which module each change appears in.

What was hard-coded	Where	Now comes from	Django field
SPEED_RANGE, MIN_SPEED, MAX_SPEED	all sul_functions	physics_constants in user_json	user_json → trace_generation
Signal indices signals[0], [1], [2]	all sul_functions	_build_target_vars from field names	main_variable, driver_signal, context
Column positions row[2], row[3]	energy parse_data	column names in target_vars	main_variable + driver_signal
CS_VERSION branches 1-10	thermostat label_event	guard expressions in events[]	user_json → events[].guard
CS='THERMO' model index 2	old teacher mi_query	Removed entirely	models[].id in user_json
CS='ENERGY' counterexample branch	old teacher get_counterexample	Removed entirely	Standard L* for all
NOISE / DELTA = 1.0	old teacher ht_s_query	self.noise	CaseStudy.noise
P_VALUE = 0.0 (hard-coded)	old teacher ht_s_query	self.p_value	CaseStudy.p_value
MI_QUERY from config.ini	old teacher mi_query	self.mi_query_flag	CaseStudy.mi_query
HT_QUERY from config.ini	old teacher ht_query	self.ht_query_flag	CaseStudy.ht_query
HT_QUERY_TYPE from config.ini	old teacher ht_query	self.ht_query_type	CaseStudy.ht_query_type
EQ_CONDITION from config.ini	old teacher + old obstable	self.eq_condition / teacher.config	CaseStudy.eq_condition
N_min from config.ini	old teacher ref_query	self.n_min	CaseStudy.n_min
Flow label vars[0][0]+vars[0][1]	old obstable to_sha	Generic flatten loop	models[].type (indirectly)
max() crash on empty table	old obstable __str__	Safe max with default=8	— (robustness fix)
CSV vs UPPAAL file passing	old teacher ref_query	resample_strategy branch	CaseStudy.resample_strategy

✓ The result is a system where adding a completely new case study requires zero Python code changes. The user fills in the web form, uploads their trace files or UPPAAL model, and the engine adapts entirely to what they provided.